[27] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019. 1, 2, 5, 7

[28] Abhishek Singh, Ayush Chopra, Ethan Garza, Emily Zhang, Praneeth Vepakomma, Vivek Sharma, and Ramesh Raskar. Disco: Dynamic and invariant sensitive channel obfuscation for deep neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12125–12135, 2021. 1

[29] Yujin Tang and David Ha. The sensory neuron as a transformer: Permutation-invariant neural networks for reinforcement learning. *Advances in Neural Information Processing Systems*, 34:22574–22587, 2021. 2

[30] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 2

[31] Hugo Touvron, Matthieu Cord, and Hervé Jégou. Deit iii: Revenge of the vit. In *European Conference on Computer Vision*, pages 516–533. Springer, 2022. 2

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1, 2

[33] Dixi Yao, Liyao Xiang, Hengyuan Xu, Hangyu Ye, and Yingqi Chen. Privacy-preserving split learning via patch shuffling over transformers. In *2022 IEEE International Conference on Data Mining (ICDM)*, pages 638–647, 2022. 1, 2, 6, 7, 8

[34] Jiahui Yu, Zirui Wang, Vijay Vasudevan, Legg Yeung, Mojtaba Seyedhosseini, and Yonghui Wu. Coca: Contrastive captioners are image-text foundation models, 2022. 1

[35] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 558–567, 2021. 1

[36] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. Deep sets. *Advances in neural information processing systems*, 30, 2017. 1, 2

[37] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. Dino: Detr with improved denoising anchor boxes for end-to-end object detection, 2022. 2

[38] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 633–641, 2017. 5

[39] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ade20k dataset. *International Journal of Computer Vision*, 127:302–321, 2019. 5

Permutation Equivariance of Transformers and Its Applications

Supplementary Material

7. Structure of Transformer

Transformer-based models are the state-of-the-art deep neural networks and have attracted great attention in both areas of computer vision and natural language processing. Models including transformer encoder blocks as their backbone, such as Bert [6], ViT [8], T2T-ViT [35], ViTGAN [12], BEiT [2] and CoCa [34], have been achieving exceeding performance in a great many tasks.

Transformer encoder blocks, as shown in Fig. 9, mainly contain two critical components: Multi-head Scaled-dot-product self-attention and a feed-forward network (MLP). Inputs are fed in the form of patches, which are usually embedding vectors for words in Bert, or for fractions of images in ViT. The relative position of patches are learned by position embeddings [32], which are injected into the model. Fig. 9 shows the main operators in a Transformer where the shortcut and the linear projection in the Attention block are left out for simplicity.

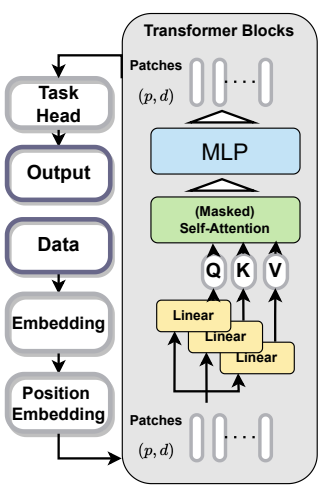

Figure 9. Transformer Encoder Block

The Transformer encoder block is denoted as Enc and the loss is ℓ. The patch embedding of a single input $\boldsymbol{X}$ is expressed as $\boldsymbol{Z}$ of shape (p, d). The first layer in the self-attention contains three parallel linear layers projecting $\boldsymbol{Z}$ to Q, K, V as

$$\boldsymbol{Q} = \boldsymbol{Z}\boldsymbol{W}_Q^\top, \tag{14}$$

$$\boldsymbol{K} = \boldsymbol{Z}\boldsymbol{W}_K^\top, \tag{15}$$

$$\boldsymbol{V} = \boldsymbol{Z}\boldsymbol{W}_V^\top. \tag{16}$$

Q, K, V are fed to the following attention operation

$$\boldsymbol{S} = Softmax(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}), \tag{17}$$

$$\boldsymbol{A} = \boldsymbol{S}\boldsymbol{V}, \tag{18}$$

where $\boldsymbol{S}$ and $\boldsymbol{A}$ are the softmax output, and the attention output, respectively.

We neglect the attention projection and the residual connection for simplicity. The part following the attention layer is the MLP layer:

$$\boldsymbol{A}_1 = \boldsymbol{A}\boldsymbol{W}_1^\top, \tag{19}$$

$$\boldsymbol{H} = a(\boldsymbol{A}_1), \tag{20}$$

$$\boldsymbol{A}_2 = \boldsymbol{H}\boldsymbol{W}_2^\top \tag{21}$$

where $\boldsymbol{A}_1, \boldsymbol{A}_2$ are the outputs of the linear layers with weights $\boldsymbol{W}_1, \boldsymbol{W}_2$, respectively, and $\boldsymbol{H}$ is the output of the element-wise activation function a which can be ReLu, Tanh, etc.

The backward propagation of Transformer encoder block is as following , we calculate the all the gradients from the final layer back to the first. Gradients are expressed as

$$\begin{aligned} \mathrm{d}l &= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}_2}^\top \mathrm{d}\boldsymbol{A}_2) \\ &= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}_2}^\top (\mathrm{d}\boldsymbol{H})\boldsymbol{W}_2^\top) + \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}_2}^\top \boldsymbol{H}\mathrm{d}(\boldsymbol{W}_2^\top)). \end{aligned}$$

The two additive terms are inspected in the following. Let's study $\boldsymbol{H}$ first:

$$\begin{aligned} \mathrm{d}l_1 &\triangleq \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}_2}^\top (\mathrm{d}\boldsymbol{H})\boldsymbol{W}_2^\top) \\ &= \mathrm{tr}(\boldsymbol{W}_2^\top \frac{\partial l}{\partial \boldsymbol{A}_2}^\top \mathrm{d}\boldsymbol{H}) \\ &= \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{A}_2}\boldsymbol{W}_2)^\top \mathrm{d}\boldsymbol{H}), \end{aligned}$$

indicating

$$\frac{\partial l}{\partial \boldsymbol{H}} = \frac{\partial l}{\partial \boldsymbol{A}_2}\boldsymbol{W}_2. \tag{22}$$

For $\boldsymbol{W}_2$,

$$\begin{aligned} \mathrm{d}l_2 &\triangleq \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}_2}^\top \boldsymbol{H}\mathrm{d}(\boldsymbol{W}_2^\top)) \\ &= \mathrm{tr}(\mathrm{d}\boldsymbol{W}_2\boldsymbol{H}^\top \frac{\partial l}{\partial \boldsymbol{A}_2}) \\ &= \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{A}_2}^\top \boldsymbol{H})^\top \mathrm{d}\boldsymbol{W}_2), \end{aligned}$$

and

$$\frac{\partial l}{\partial \boldsymbol{W}_2} = \frac{\partial l}{\partial \boldsymbol{A}_2}^\top \boldsymbol{H}. \tag{23}$$

For $\boldsymbol{A}_1$:

$$\begin{aligned}
\mathrm{d}l_1 &= \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{H}}^\top \mathrm{d}\boldsymbol{H}) \\
&= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{H}}^\top \mathrm{d}(a(\boldsymbol{A}_1))) \\
&= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{H}}^\top a'(\boldsymbol{A}_1) \odot \mathrm{d}\boldsymbol{A}_1)) \\
&= \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{H}} \odot a'(\boldsymbol{A}_1))^\top \mathrm{d}\boldsymbol{A}_1),
\end{aligned}$$

by Eq. 22, we have

$$\frac{\partial l}{\partial \boldsymbol{A}_1} = \frac{\partial l}{\partial \boldsymbol{A}_2} \boldsymbol{W}_2 \odot a'(\boldsymbol{A}_1). \tag{24}$$

Similarly, we calculate the gradients of $\boldsymbol{A}$ and $\boldsymbol{W}_1$:

$$\frac{\partial l}{\partial \boldsymbol{A}} = \frac{\partial l}{\partial \boldsymbol{A}_1} \boldsymbol{W}_1, \tag{25}$$

$$\frac{\partial l}{\partial \boldsymbol{W}_1} = \frac{\partial l}{\partial \boldsymbol{A}_1}^\top \boldsymbol{A}. \tag{26}$$

In the attention operation:

$$\begin{aligned}
\mathrm{d}l_3 &\triangleq \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}}^\top \mathrm{d}\boldsymbol{A}) \\
&= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{A}}^\top (\mathrm{d}\boldsymbol{S})\boldsymbol{V}) + \mathrm{tr}(\frac{\partial l}{\partial A}^\top \boldsymbol{S}\mathrm{d}\boldsymbol{V}) \\
&= \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{A}}\boldsymbol{V}^\top)^\top \mathrm{d}\boldsymbol{S}) + \mathrm{tr}((\boldsymbol{S}^\top \frac{\partial l}{\partial \boldsymbol{A}})^\top \mathrm{d}\boldsymbol{V}),
\end{aligned}$$

and

$$\frac{\partial l}{\partial \boldsymbol{S}} = \frac{\partial l}{\partial \boldsymbol{A}} \boldsymbol{V}^\top, \tag{27}$$

$$\frac{\partial l}{\partial \boldsymbol{V}} = \boldsymbol{S}^\top \frac{\partial l}{\partial \boldsymbol{A}}. \tag{28}$$

First, for $\boldsymbol{V} = \boldsymbol{Z}\boldsymbol{W}_V^\top$:

$$\begin{aligned}
\mathrm{d}l_4 &\triangleq \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{V}}^\top \mathrm{d}\boldsymbol{V}) \\
&= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{V}}^\top (\mathrm{d}\boldsymbol{Z})\boldsymbol{W}_V^\top) + \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{V}}^\top \boldsymbol{Z}\mathrm{d}\boldsymbol{W}_V^\top).
\end{aligned}$$

Similarly, the gradients of $\boldsymbol{Z}$ and $\boldsymbol{W}_V$ are:

$$\frac{\partial l}{\partial \boldsymbol{Z}} = \frac{\partial l}{\partial \boldsymbol{V}} \boldsymbol{W}_V, \tag{29}$$

$$\frac{\partial l}{\partial \boldsymbol{W}_V} = \frac{\partial l}{\partial \boldsymbol{V}}^\top \boldsymbol{Z}. \tag{30}$$

Now we focus on $\boldsymbol{S} = Softmax(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}})$:

$$\begin{aligned}
\mathrm{d}l_5 &\triangleq \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{S}}^\top \mathrm{d}\boldsymbol{S}) \\
&= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{S}}^\top (diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S})\mathrm{d}(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}})) \\
&= \mathrm{tr}(((diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S})^\top \frac{\partial l}{\partial \boldsymbol{S}})^\top \mathrm{d}(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}})),
\end{aligned}$$

and thus

$$\frac{\partial l}{\partial \boldsymbol{Q}} = \frac{1}{\sqrt{d}}((diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S})^\top \frac{\partial l}{\partial \boldsymbol{S}})\boldsymbol{K}, \tag{31}$$

$$\frac{\partial l}{\partial \boldsymbol{K}} = \frac{1}{\sqrt{d}}((diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S})^\top \frac{\partial l}{\partial \boldsymbol{S}})^\top \boldsymbol{Q}. \tag{32}$$

And similarly the gradients of $\boldsymbol{W}_Q$ and $\boldsymbol{W}_K$ are:

$$\frac{\partial l}{\partial \boldsymbol{W}_Q} = \frac{\partial l}{\partial \boldsymbol{Q}}^\top \boldsymbol{Z}, \tag{33}$$

$$\frac{\partial l}{\partial \boldsymbol{W}_K} = \frac{\partial l}{\partial \boldsymbol{K}}^\top \boldsymbol{Z}. \tag{34}$$

8. Alg. on Permuted Training

Our permuted training is described by pseudo code in Alg. 1. It should be noted that the permutation takes place not on the dimension of 'batches' but on the rest two dimensions. Taking ViT for example, each image is transformed into a (p, d) matrix representing p patches, and each patch denotes a fraction of the image. Each fraction is embedded into a d-dimensional vector.

We further provide a toy example. Let $\boldsymbol{Z}$ of shape $(3, 4)$ and the row shuffle matrix $\boldsymbol{P}_R$ be

$$\boldsymbol{Z} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix} \quad \boldsymbol{P}_R = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

The row permuted feature is

$$\boldsymbol{P}_R\boldsymbol{Z} = \begin{pmatrix} 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 1 & 2 & 3 & 4 \end{pmatrix}.$$

Column permutation is performed in a similar way. Note that permutation matrices are orthogonal, i.e., $\boldsymbol{P}_R^{-1} = \boldsymbol{P}_R^\top$ and $\boldsymbol{P}_C^{-1} = \boldsymbol{P}_C^\top$.